System Overview

Agromind – Smart Agro-Aqua Intelligence Platform

Agromind is a distributed AI-powered smart farming platform designed to unify agriculture, aquaculture, environmental analytics, energy optimization, and cognitive intelligence into a single integrated ecosystem.

The system follows a **layered hybrid architecture** composed of:

- Web Application (Next.js)
- Mobile Application (Flutter)
- Central Business Backend (.NET Core)
- AI Intelligence Backend (FastAPI – Python)
- Cognitive LLM Backend
- PostgreSQL + TimescaleDB Data Layer

The architecture ensures scalability, modularity, and clear separation between presentation, business logic, predictive intelligence, and reasoning capabilities.

System Architecture Layers

1. **Presentation Layer**
   - **Web Application (Next.js)**

     The main user interface of the platform Provides:

     1. Role-based dashboards
     2. Farm & pond management
     3. Analytics visualization
     4. AI predictions display
     5. Admin control panel

     The frontend communicates securely with the .NET backend via REST APIs.

- **Mobile Application (Flutter)**

  Built for field usage by farmers and fishers Features:

  1. Manage farms & ponds
  2. Monitor crop and fish health
  3. Receive alerts & notifications
  4. View AI predictions
  5. Interact with chatbot

  The mobile app connects to the same .NET backend APIs used by the web app.

Role-Based Access Control

The platform implements secure authentication and authorization using JWT.

Roles:

Customer (Farmer / Fisher)

Access to:

- Customer Dashboard
- Farm management
- Pond management
- AI recommendations
- Chatbot support
- Alerts & monitoring

Admin

Access to:

- Admin Dashboard
- User management
- System analytics

- Platform monitoring
- Logs & reports
- Configuration controls

Role permissions are enforced at the backend level.

---

2. **Application Layer – Main Backend (.NET Core)**

This backend acts as the central controller of the system.

Responsibilities:

- Authentication & Authorization (JWT)
- Role management
- CRUD operations
- File & image storage
- Workflow orchestration
- Data validation
- Communication with AI backend
- Communication with LLM backend
- Logging & monitoring
- Admin analytics aggregation

The .NET backend does not run AI models directly.
It forwards structured requests to the AI backend and processes responses.

---

3. **AI Intelligence Backend (FastAPI – Python)**

This backend contains all machine learning, deep learning, forecasting, and optimization models.

Internally, it is modular and structured into four intelligence domains:

Aquaculture Intelligence Module

- Fish disease detection
- Water quality forecasting
- Fish growth prediction
- Feeding optimization
- Harvest planning

Agriculture Intelligence Module

- Plant disease detection
- Nutrient deficiency analysis
- Crop recommendation
- Fertilizer suggestion
- Yield prediction
- Soil quality evaluation

Weather & Environmental Intelligence Module

- Weather forecasting
- Drought & flood prediction
- Sensor anomaly detection
- Environmental risk scoring

Energy & Smart Resource Optimization Module

- Energy consumption forecasting
- Pump scheduling optimization
- Solar energy prediction
- Cost-aware resource planning

4. **Cognitive Intelligence Backend (LLM Service)**

This backend runs a locally hosted Large Language Model (LLaMA-based).

Responsibilities:

- Conversational AI chatbot
- Advisory support for farmers & fishers
- Alert explanation
- Cross-domain reasoning
- Decision interpretation
- Knowledge summarization

This layer integrates predictions from all AI modules and transforms them into:

- Human-readable explanations
- Actionable recommendations
- Strategic planning insights

It acts as the reasoning layer of the entire platform.

---

5. **Data Layer**

PostgreSQL

Used for:

- Users
- Farms & ponds
- Crop records
- Logs
- System data

TimescaleDB Extension

Used for:

- Sensor readings

- Environmental metrics
- Growth tracking
- Energy usage data
- Time-series analytics

This ensures high-performance structured and time-based data management.

System Communication Flow

1. User interacts with Web or Mobile app
2. Request sent to .NET backend
3. Backend validates role & data
4. If AI prediction required → forwarded to AI Intelligence backend
5. If explanation required → forwarded to LLM backend
6. Responses aggregated
7. Stored in database
8. Final result returned to user dashboard

Distribution Summary

Layer	Technology	Role
Web App	Next.js	User Interface
Mobile App	Flutter	Field Interface
Main Backend	ASP.NET Core	Business Logic & Orchestration
AI Backend	FastAPI	ML/DL & Optimization Models
LLM Backend	FastAPI	Cognitive Reasoning
Database	PostgreSQL + TimescaleDB	Structured & Time-Series Storage